0x – Settler SELECT Action

Smart Contract Security Assessment

September 11, 2026



ABSTRACT

Dedaub was commissioned to perform a security audit of the 0x Settler SELECT action.

BACKGROUND

The 0x Settler SELECT action extends the Settler's EVM settlement layer with on-chain, ordered route selection. SELECT accepts several candidate routes, each with its own minimum-output target, and tries them in order inside revertible self-calls. The first candidate whose output in the scoring token meets its target is committed; every losing candidate is rolled back, and if none qualifies the transaction reverts. A per-trial gas cap keeps a failing candidate from starving those behind it and, together with a pre-call gas check, guarantees that the transaction's gas limit cannot influence which route is chosen. The off-chain service remains responsible for computing the candidates, their ordering and targets, and the overall slippage bound; the contract's role is purely to execute the first acceptable route and enforce the final bound on-chain.

SETTING & CAVEATS

This audit report mainly covers PR#641 of the 0x Settler protocol containing changes between commits `e80cfb0234cc7153abb34e63ad80e9ea2cf27dff` and `f358d811483fdad25486b34b5ca20e4d4393eaf6`. Fix review was performed on commit `fb24d5300fdc0a31bf8d961e8e145a405e8bf92b`.

Audit Start Date:	**Sep 9, 2026**
Report Submission Date:	**Sep 11, 2026**
Fix Review Date:	**Nov 4, 2026**

2 auditors worked on the following contracts:

```
src/
├── chains/
│   ├── Mainnet/
│   │   ├── Common.sol
├── core/
│   ├── Permit2PaymentAbstract.sol
│   ├── Permit2Payment.sol
│   ├── Select.sol
│   ├── SettlerErrors.sol
├── ISettlerActions.sol
├── SettlerBase.sol
```

The audit's main target is security threats, i.e., what the community understanding would likely call "hacking", rather than the regular use of the protocol. Functional correctness (i.e. issues in "regular use") is a secondary consideration. Typically it can only be covered if we are provided with unambiguous (i.e. full-detail) specifications of what is the expected, correct behavior. In terms of functional correctness, we often trusted the code's calculations and interactions, in the absence of any other specification. Functional correctness relative to low-level calculations (including units, scaling and quantities returned from external protocols) is generally most effectively done through thorough testing rather than human auditing.

VULNERABILITIES & FUNCTIONAL ISSUES

This section details issues affecting the functionality of the contract. Dedaub generally categorizes issues according to the following severities, but may also take other considerations into account such as impact or difficulty in exploitation:

Category	Description
CRITICAL	Can be profitably exploited by any knowledgeable third-party attacker to drain a portion of the system's or users' funds OR the contract does not function as intended and severe loss of funds may result.
HIGH	Third-party attackers or faulty functionality may block the system or cause the system or users to lose funds. Important system invariants can be violated.
MEDIUM	Examples: • User or system funds can be lost when third-party systems misbehave. • DoS, under specific conditions. • Part of the functionality becomes unusable due to a programming error.
LOW	Examples: • Breaking important system invariants but without apparent consequences. • Buggy functionality for trusted users where a workaround exists. • Security issues which may manifest when the system evolves.

Issue resolution includes "dismissed" or "acknowledged" but no action taken, by the client, or "resolved", per the auditors.

CRITICAL SEVERITY:

[No critical severity issues]

HIGH SEVERITY:

[No high severity issues]

MEDIUM SEVERITY:

[No medium severity issues]

LOW SEVERITY:

ID	Description	STATUS
L1	Semantic inconsistency of `CHECK_SLIPPAGE` can lead to unexpected candidate being selected	**ACKNOWLEDGED**

Comments: Developers acknowledge this; documentation (`ISettlerActions.sol:23-236`) notes that `SELECT` candidates should not contain `CHECK_SLIPPAGE`, but it will not be checked at runtime.

`Select::_runActions` deliberately runs candidates against a fresh zeroed `AllowedSlippage` , since the score must be measured on Settler's balance.

`src/core/Select.sol::_runActions:`

```
function _runActions(bytes[] calldata actions) private {
    // A nested `CHECK_SLIPPAGE` no-ops against this zeroed struct in
taker-submitted Settler.
    // The other flavors do not dispatch it, so there it reverts only its own
trial.
```

```
    AllowedSlippage memory noSlippage;
    ...
        if (tooShort || !_dispatch(i, action, data, noSlippage)) {
            revertActionInvalid(i, action, data);
        }
```

This causes `CHECK_SLIPPAGE` to have different semantics inside a SELECT compared to its top-level behavior. At top level, `CHECK_SLIPPAGE(bool transferExactLimit)` is a real settlement step, not just an assertion: it reads the taker's `AllowedSlippage`, reverts `TooMuchSlippage` if Settler's buyToken balance is below minAmountOut and transfers the output to recipient. But inside SELECT with a zeroed `AllowedSlippage`, `_checkSlippageAndTransfer` hits `minAmountOut == 0 && buyToken == 0` and becomes a no-op.

Funds are not at risk from the no-op alone: the taker's real `AllowedSlippage` is untouched inside a trial, and `_execute` still runs `_checkSlippageAndTransfer` with it after SELECT commits, so the end-of-run bound and payout still happen. The problem is that the inconsistent behavior of `CHECK_SLIPPAGE` could cause a different candidate to be unexpectedly selected. Consider, for instance, a candidate aimed to "pay the taker exactly the quoted amount, route the surplus elsewhere", expressed as
`candidates[0] = [ swap 5 bps pool (fills 2,010), CHECK_SLIPPAGE(true), BASIC sweep remainder -> SINK ]  targets[0] = 1_950e6`

This would be correct at the top level, but fails inside SELECT because `CHECK_SLIPPAGE` no-ops and the BASIC sweeps everything, causing the slippage check after the execution to fail.
Now compose this with a candidate without CHECK_SLIPPAGE, e.g.:
`candidates[1] = [ swap 30 bps pool (fills 1,985) ]  targets[1] = 0`

Then SELECT will succeed, but will execute the second candidate, even though top-level simulation shows that the first candidate is valid.

To avoid such semantic inconsistencies we recommend forbidding `CHECK_SLIPPAGE` actions in `Select.sol::_runActions`.

L2	`NATIVE_CHECK` inside a SELECT candidate cannot enforce its `msg.value` bound	**ACKNOWLEDGED**

Comments: Developers acknowledge this; documentation (`ISettlerActions.sol:235-236`) notes that `SELECT` candidates should not contain `NATIVE_CHECK`, but it will not be checked at runtime.

`NATIVE_CHECK(deadline, msgValue)` is the guard on native-asset sells: it reverts `SignatureExpired` if the quote has expired and `MsgValueMismatch` if the taker attached more ETH than the quote was built for. This is useful because over-attached ETH becomes WETH the pool does not consume, the end-of-run check moves only buyToken, and the leftover WETH sits in Settler until the next caller's `BASIC` sweeps it.

Inside a `SELECT`, however, trials are self- `CALL`s with zero value, so inside a candidate `callvalue() == 0` and `msg.value > msgValue` can never be true; essentially only the deadline half of the check executes and over-funding is unprotected.

Consider, for instance, a route `[NATIVE_CHECK(deadline, 1 ETH), wrap 100 %, sell 100 %].` If the taker accidentally attaches 10 ETH, but the sell action cannot use all of it (e.g., a price-limited route), the `NATIVE_CHECK` will not protect it and the remaining funds will sit in Settler.

To avoid intended checks silently succeeding we recommend forbidding `NATIVE_CHECK` actions in `Select.sol::_runActions`.

OTHER / ADVISORY ISSUES:

This section details issues that are not thought to directly affect the functionality of the project, but we recommend considering them.

ID	Description	STATUS
A1	Auxiliary functions changing meaning inside a trial	**RESOLVED**

Comments: `ISettlerActions.sol:235` now documents that "*Candidates run with Settler as the caller and no call value*", as of e1bb854.

Inside a trial `msg.sender == address(this)`, causing `_isForwarded()` to become `false` and `_operator()` to become the Settler. No `_dispatch`-reachable action depends on these today (`fillRfqOrderSelfFunded` passes `isForwarded = false` explicitly; all other consumers are VIP-only), so this is not a direct issue. Still, a future non-VIP action that reads either `_isForwarded/_operator` would silently misbehave inside `SELECT` (similarly to `NATIVE_CHECK` misbehaving due to `msg.value` see L2).

We recommend carefully documenting which parts of the code are affected by `SELECT` to ensure that future versions do not introduce subtle misbehaviours.

A2	The gas-starvation boundary is untested	**RESOLVED**

Comments: The developers have added the recommended tests in `SelectUnitTest.t.sol`, with commit 5552eaa.

`_SELECT_OVERHEAD_GAS = 8192` is the soundness condition for skipping a failed trial: it must exceed the gas spent between the `gas()` read and the `CALL`, or a starved trial can be misclassified as fully funded and its OOG failure skipped. Although this is an important and highly non-trivial constant, nothing in the suite asserts it.

An instrumented run during this audit measured that gap at 382 gas (solc 0.8.34 / 2,000 runs / osaka), so the overestimation margin seems more than enough. Still, adding concrete tests asserting the bound is recommended as gas consumption is often sensitive to the exact compiler version.		
A3	Compiler bugs	**INFO**
The code is compiled with Solidity `0.8.34`. This version has some known bugs, which we do not believe affect the correctness of the contracts.		

DISCLAIMER

The audited contracts have been analyzed using automated techniques and extensive human inspection in accordance with state-of-the-art practices as of the date of this report. The audit makes no statements or warranties on the security of the code. On its own, it cannot be considered a sufficient assessment of the correctness of the contract. While we have conducted an analysis to the best of our ability, it is our recommendation for high-value contracts to commission several independent audits, a public bug bounty program, as well as continuous security auditing and monitoring through Dedaub Security Suite.

ABOUT DEDAUB

Dedaub offers significant security expertise combined with cutting-edge program analysis technology to secure some of the most prominent protocols in DeFi. The founders, as well as many of Dedaub's auditors, have a strong academic research background together with a real-world hacker mentality to secure code. Protocol blockchain developers hire us for our foundational analysis tools and deep expertise in program analysis, reverse engineering, DeFi exploits, cryptography and financial mathematics.